实验 4：前后端开发实验指导

一、实验目标

1. 基于实验 1 的数据集特性、实验 2 训练的模型及实验 3 优化后的模型，构建 AI 驱动型应用的前后端架构，形成“数据-模型-应用”完整闭环；

2. 掌握前端 AI 交互界面开发，实现与用户的数据交互（如图像上传、文本输入），界面设计需适配实验 1 的多类型数据（图像/文本/传感器数据）；

3. 完成后端 AI 服务 API 开发，集成实验 3 优化后的模型，实现模型推理、数据处理的服务化封装；

4. 掌握前后端数据流设计、API 规范与异常处理逻辑，解决跨平台兼容性与分布式任务协调问题，强化工程化落地能力。

二、实验环境

硬件：

-基础配置：（CPU ≥ 4 核，内存 ≥ 8GB），浏览器（Chrome/Firefox/Safari，用于跨平台测试）；

软件：

- 前端：Node.js 24+、React 19+（或 Vue 3+）、Ant Design（UI 组件库）、Axios（HTTP 请求）；

- 后端：PyCharm（Python 3.14+）、Anaconda（Python 3.12）、FastAPI 0.16+（API 框架）、Uvicorn（ASGI 服务器）、Celery 5.6+（异步任务）；

- 部署工具：Docker 29.0+（容器化）、Postman（API 测试）；

- 关联依赖：实验 3 优化模型（量化 ResNet/蒸馏 BERT）、实验 1 预处理逻辑（复用数据清洗函数）。

三、实验学时与安排

共 6 学时，分 2 个阶段完成：

- 阶段 1：前端开发（3 学时）设计 AI 交互界面，适配实验 1 的多类型数据输入；

- 阶段 2：后端开发与集成（3 学时）开发 API 服务，集成实验 3 优化模型，实现前后端协同。

四、实验步骤

阶段 1：前端开发（4.1 模块）

目标：开发支持多类型数据交互的 AI 应用前端界面，实现用户输入、结果展示与交互逻辑，数据流转需匹配后端 API 设计。

任务 4.1.1 前端架构设计与环境搭建

1. 项目初始化（如使用 React）。

2. 页面架构设计（需关联实验 1 的多数据类型）：

- 核心页面：「图像分类页」（适配实验 1 的 CIFAR-10 图像数据）、「文本分类页」（适配实验 1 的技术博客文本）、「多模态匹配页」（适配实验 1 的图文对数据）；

- 公共组件：导航栏、数据输入区（文件上传/文本框）、结果展示区（分类标签/置信度）、异常提示组件。

任务 4.1.2 AI 交互界面开发

1. 界面组件实现

2. 图像分类页、文本分类页、多模态匹配页开发

任务 4.1.3 跨平台兼容性优化

1. 响应式设计：使用 Ant Design 的 Grid 组件适配不同屏幕尺寸（手机/平板/PC）；

2. 浏览器兼容性测试：覆盖 Chrome（主力）、Safari（移动端/iOS WebKit 核心，桌面端份额亦居第二）和 Firefox（多样性验证）；Firefox 在当前桌面生态中份额较小，测试重点可集中于 Chrome 与 Safari 的 WebKit 差异。

阶段 2：后端开发与集成（4.2 模块）

目标：开发后端 AI 服务 API，集成实验 3 优化后的模型，实现推理服务化，设计数据流与异常处理逻辑，支持前端交互。

任务 4.2.1 后端服务架构设计与环境搭建

1. 项目初始化

2. 目录结构设计

任务 4.2.2 AI 服务 API 开发（核心功能）

1. 模型加载（关联实验 3 优化模型）

2. API 路由实现（RESTful 规范）

3. 应用 API 开发（关联实验 3）

任务 4.2.3 工作流编排与分布式任务协调

1. 多模型协同工作流（以“图文匹配”为例）：

- 流程：前端上传图像+文本→后端先调用图像分类 API→再调用文本分类 API→对比类别一致性→返回匹配结果；

- 实现：通过 FastAPI 依赖注入串联多个 API 逻辑。

2. 异步任务处理（分布式协调）：

- 针对大文件/复杂推理任务（如实验 1 的高分辨率图像），用 Celery 实现异步处理；

任务 4.2.4 前后端集成与测试

1. 跨域配置（解决前端请求限制）

2. 端到端测试

五、重点与难点说明

重点关注

1. 前后端数据流设计：

- 核心流程：前端用户输入（图像/文本）→ 表单校验→ Axios POST 请求→ 后端接收（FastAPI）→ 数据预处理（复用实验 1 逻辑）→ 实验 3 模型推理→ 结果 JSON 返回→ 前端渲染展示；

- 数据格式规范：图像用 multipart/form-data，文本用 application/json，结果统一格式为{"code": 状态码, "result": 数据, "msg": 提示}。

2. API 设计规范：

- RESTful 风格：URL 使用名词（如/api/v1/image），方法对应动作（GET 查询/POST 提交）；

- 版本控制：API 路径包含版本（如 v1），便于后续迭代；

- 文档化：用 FastAPI 自动生成 API 文档（访问 http://localhost:8000/docs），包含参数说明、返回示例。

3. 异常处理：

- 前端：捕获 Axios 请求错误（网络错误、500 错误），显示用户友好提示（如“图像格式错误，请上传 PNG/JPG”）；

- 后端：定义异常类型（输入错误、模型错误、权限错误），返回标准化错误码

（400 参数错、404 路径错、500 服务器错）。

难点突破

1. 跨平台兼容性

- 前端：使用 Flex/Grid 布局替代绝对定位，避免浏览器样式差异；目标浏览器已原生支持 ES6+，Polyfill 仅在覆盖非常规低版本引擎时考虑；

- 后端：API 返回数据避免使用浏览器敏感字符（如特殊符号），跨域配置需明确 allow_origins（开发环境允许 localhost，生产环境限制域名）。

2. 分布式任务协调：

- 异步任务：用 Celery+Redis 处理耗时任务（如大图像推理），前端通过轮询或 WebSocket 获取结果；

- 任务调度：对多模型协同任务（如图文匹配），用工作流引擎（如 Prefect）定义依赖关系，确保执行顺序（先图像分类→再文本分类）。

六、实验报告要求

1. 说明小组成员构成，明确每个成员完成的任务；

2. 架构设计文档：

- 前后端架构图：标注核心组件（前端页面/组件、后端 API/模型）及数据流方向；

- 关联分析：说明前端如何适配实验 1 的多数据类型，后端如何集成实验 3 优化模型。

3. API 文档：

- 列出所有 API 的路径、方法、参数、返回值（附实验 1 测试数据的调用示例）；

- 异常处理说明：包含错误码表及对应场景。

4. 测试报告：

- 跨平台测试结果：在 3 种浏览器中的功能测试记录（通过/不通过及修复方案）；

- 性能测试：记录实验 3 优化模型的推理耗时（前端展示延迟≤2 秒为合格）。

5. 问题与解决方案：记录开发中遇到的典型问题（如跨域报错、模型推理延迟）及解决方法。

七、参考资料

1. [React 官方文档]

(https://react.dev/)

2. [FastAPI 文档]

(https://fastapi.tiangolo.com/)；

3. [RESTful API 设计指南]

(https://restfulapi.net/)；

4. [Celery 异步任务教程]

(https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html)